

apk-analyzer — Technical Architecture & Library Rationale

Why Specific Libraries Were Chosen & Viva / Interview FAQ in Hinglish

1. Library Selection & Rejection Matrix

The table below details why specific Python libraries and frameworks were selected for the apk-analyzer static reverse-engineering pipeline and why alternative tools were rejected.

Component / Layer	Chosen Library / Approach	Rejected Alternatives	Technical Justification & Why Only This Tool
APK Ingestion & Hashing	Python stdlib (zipfile, hashlib)	patool, shutil unpack	Zero external dependencies. Runs directly in memory without installing C binaries or external tools.
Binary Manifest Parser	Custom Pure-Python AXML Parser (axml.py)	apktool, androguard, pyaxmlparser	Apktool requires Java JRE and subprocess spawning (adds 3-5s latency). Androguard has heavy legacy dependencies (networkx, pyasn1). Custom parser decodes AXML in under 5ms.
Data Schema & Validation	Pydantic v2	dataclasses, raw dicts	Guarantees strict schema validation, risk score constraints (0-100), and automated JSON output matching security standards.
Bytecode & Entropy Engine	Pure Python math & regex (re)	capstone, javassist, dexlib2	Calculates Shannon Entropy (H) directly on DEX bytes without native C-extension compilation dependencies.
PDF Generation	ReportLab 4.2	weasyprint, pdfkit (wkhtmltopdf)	Pure Python vector PDF engine. Does not require headless Chrome or external system webkit binaries.
Web Interface	Single-file HTML5 + Tailwind + JSZip	React, Vue, Angular, Next.js	Zero build tools (npm/node_modules). Operates offline in any browser; JSZip enables client-side zip parsing.

2. Frequently Asked Questions (Viva / Interview Q&A; in Hinglish)

Q1: Is project me androguard ya apktool kyun use nahi kiya?

Answer: apktool ko chalane ke liye system me Java (JRE) installed hona zaroori hota hai aur wo background me subprocess spawn karta hai jisse analysis bohot slow (~3-5 seconds) ho jaati hai. androguard me bohot saari heavy external dependencies hoti hain. Humne ek Pure Python Binary AXML Parser (axml.py) likha hai jo directly APK ke AndroidManifest.xml bytes ke String Pool aur Resource IDs ko 5 milliseconds me decode kar leta hai bina kisi external tool ya Java setup ke.

Q2: Dynamic Analysis (Live Sandbox Execution) kyun nahi kiya, sirf Static Analysis kyun?

Answer: Dynamic analysis me malware ko real device ya emulator pe run karna padta hai jo risky hota hai, battery/cpu intensive hota hai, aur malware sandbox evasion (jaise isDebuggerConnected() check karna) se chhup sakta hai. Static & Heuristic Analysis fast hota hai, safe hota hai (code execute hi nahi hota), aur application ki saari permissions, hardcoded C2 IPs, dynamic class loaders (DexClassLoader), aur debug certs ko bina run kiye instantly spot kar leta hai.

Q3: Shannon Entropy calculation se packing aur malware kaise detect hota hai?

Answer: Normal uncompressed DEX bytecode ka Shannon Entropy score around 4.0 to 6.2 hota hai kyunki code me repeated structure hoti hai. Lekin jab malware author code ko pack, encrypt ya obfuscate karta hai (jaise Qihoo 360, Bangle), to byte randomness badh jaati hai aur Entropy 7.4 se 8.0 ho jaati hai. Agar entropy >= 7.4 milti hai, to hamara engine ise Extremely High / Packed Payload flag kar deta hai.

Q4: Risk Score 0 se 100 kaise calculate hota hai?

Answer: Risk Score har phase ke findings ke weights ko calculate karke aggregation karta hai: Critical Findings (C2 endpoints, Accessibility Service abuse, Banking Trojan Overlay signature, DexClassLoader + Disguised assets) ko +25 to +35 points; High Findings (Debug certs, Unprotected Boot Receivers, Shell execution) ko +20 points; Dangerous Permissions ko +5 points per permission (capped at 25); aur Packing/High Entropy ko +15 points. Score 0-39 ko Safe, 40-69 ko Suspicious, aur 70-100 ko Malicious categorize kiya jata hai.

Q5: Web Interface offline kaise kaam karta hai?

Answer: Web UI ko single-file HTML5 format me Tailwind CSS CDN aur inline JavaScript logic ke saath banaya gaya hai. Ye user ke browser me run hota hai, jisme JSZip engine binary APKs ko client-side unpack karta hai aur simulated/live progress logging ke sath interactive risk meter gauge aur JSON report generation render karta hai.

Q6: What are dangerous permission combinations in Android security?

Answer: Single permissions dangerous ho sakti hain, lekin combinations zyada lethal hoti hain: 1) Banking Trojan Overlay: RECEIVE_BOOT_COMPLETED + SYSTEM_ALERT_WINDOW (boot hote hi overlay launch karke bank credentials phish karna). 2) Spyware Exfiltration: INTERNET + READ_CONTACTS/RECORD_AUDIO + SEND_SMS. 3) SMS 2FA Interception: RECEIVE_SMS + SEND_SMS + INTERNET (bank OTPs steal karke remote server pe bhejna).